Course: Parallel and Distributed Computing (ECOM 6339)

Student Name: Dalya Awad Mohammad Hassan

Student ID: 220253121

Assignment: Assignment 01

Date: 17 October 2026

Task 1: Amdahl's Law

Consider a parallel program with serial fraction s = 0.05. Complete the following:

1. Compute the speedup S(p) for p = 1, 2, 4, 8, 16, 64, 1024.

Amdahl's law: $Speedup\ S(p) = \frac{1}{s+\frac{(1-s)}{p}}$

where:

- S(p) = speedup using p processors
- s = serial fraction of the program ➔ s = 0.05
- 1 - s = parallel fraction ➔ 1 – s = 0.95
- p = number of processors ➔ p = 1, 2, 4, 8, 16, 64, 1024

$$S(1) = \frac{1}{0.05 + 0.95} = 1$$

$$S(2) = \frac{1}{0.05 + \frac{0.95}{2}} = \frac{1}{0.525} = 1.904762$$

$$S(4) = \frac{1}{0.05 + \frac{0.95}{4}} = \frac{1}{0.2875} = 3.478261$$

$$S(8) = \frac{1}{0.05 + \frac{0.95}{8}} = \frac{1}{0.16875} = 5.925926$$

$$S(16) = \frac{1}{0.05 + \frac{0.95}{16}} = \frac{1}{0.109375} = 9.142857$$

$$S(64) = \frac{1}{0.05 + \frac{0.95}{64}} = \frac{1}{0.06484375} = 15.421687$$

$$S(1024) = \frac{1}{0.05 + \frac{0.95}{1024}} = \frac{1}{0.05092773} = 19.635666$$

2. Derive the maximum possible speedup as p → ∞.

As p → ∞, the parallel effect on the execution time goes to zero:

$$\lim_{p\to\infty} \frac{1-s}{p} = 0$$

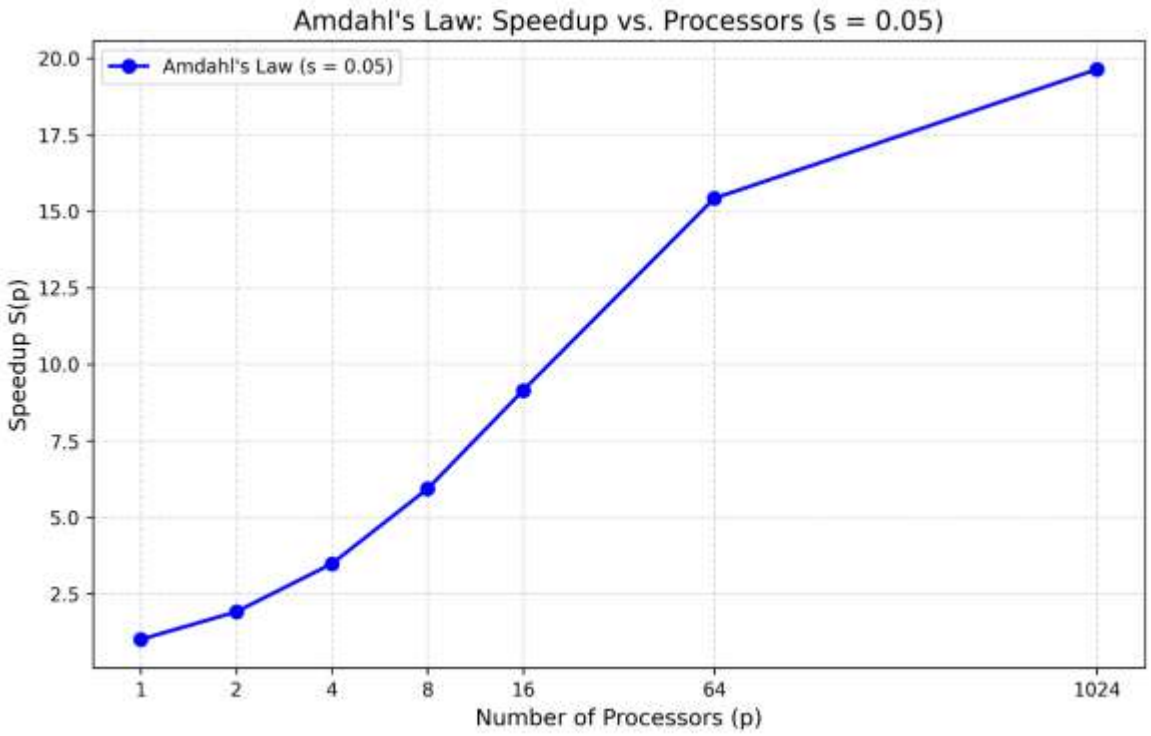
So

$$S_{\max} = \lim_{p\to\infty} S(p) = \frac{1}{s+0} = \frac{1}{s}$$

And for s = 0.05 ➔ The max speedup $= \frac{1}{0.05} = 20$

This means even with an unlimited number of processors, a program with a 5% serial fraction cannot exceed a theoretical speedup of 20 under the standard Amdahl model. This is because the serial portion cannot be accelerated by adding processors.

3. Plot S(p) vs. p (log scale for p) using matplotlib or gnuplot.



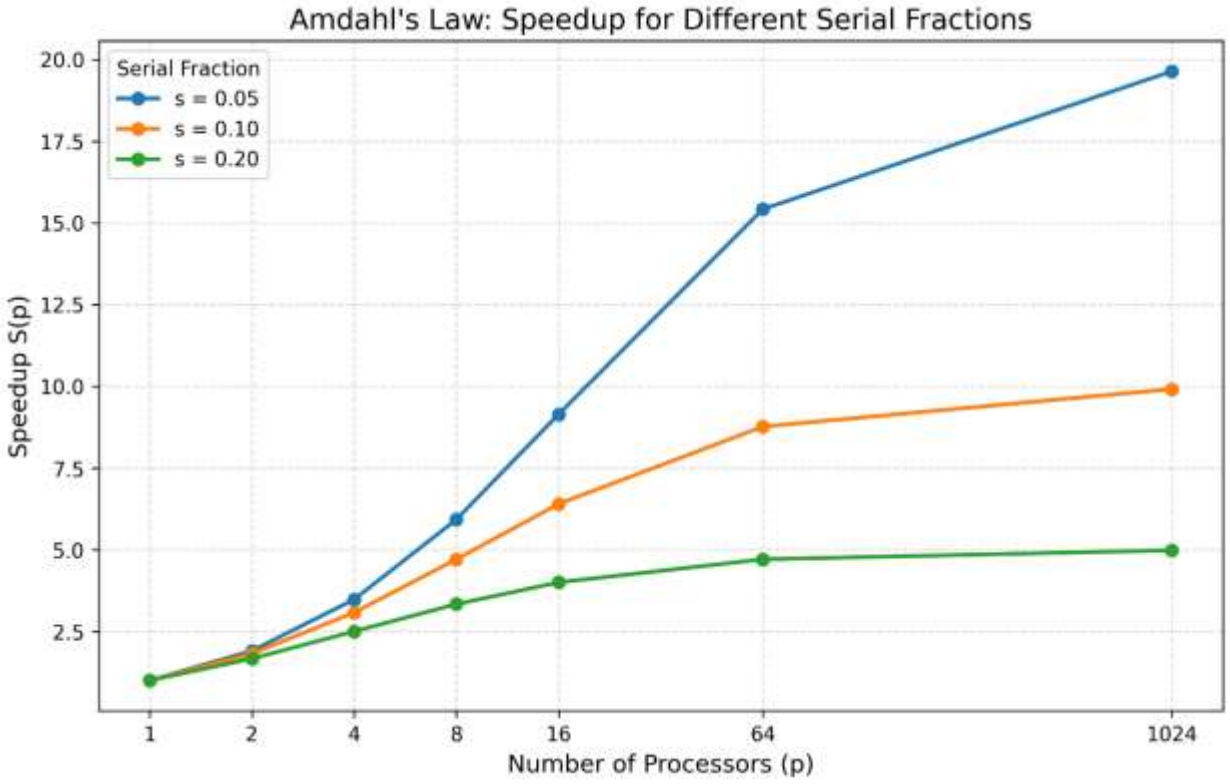
The graph illustrates how the number of processors affects the theoretical speedup according to Amdahl's Law, assuming a serial fraction s=0.05. Speedup increases as more processors are added, but the rate of improvement gradually decreases due to the serial portion of the program. At 1024 processors, the theoretical speedup reaches approximately 19.64, reaching the maximum possible speedup of 20.

Dalya Mohammad Hassan
220253121

4. Repeat for s = 0.10 and s = 0.20 and plot all three on the same graph.

```
Serial Fraction: s = 0.10
Processors (p) | Speedup S(p)
--------------------------------
             1 | 1.0000
             2 | 1.8182
             4 | 3.0769
             8 | 4.7059
            16 | 6.4000
            64 | 8.7671
          1024 | 9.9129

Serial Fraction: s = 0.20
Processors (p) | Speedup S(p)
--------------------------------
             1 | 1.0000
             2 | 1.6667
             4 | 2.5000
             8 | 3.3333
            16 | 4.0000
            64 | 4.7059
          1024 | 4.9805
```



The plot shows that all three curves begin at S(1)=1 and gradually go toward their respective theoretical maximum speedups of 20, 10, and 5.

Dalya Mohammad Hassan
220253121

5. Explain the difference in scaling behavior.

Amdahl's Law demonstrates that increasing the processor count improves performance, but the improvement becomes smaller because the serial portion of the program remains unchanged.

For a serial fraction s=0.05, the program achieves a speedup of approximately 19.64 using 1024 processors, approaching the theoretical limit of 20. When the serial fraction increases to s=0.10, the maximum possible speedup decreases to 10. At a serial fraction s=0.20, the maximum theoretical speedup is only 5, regardless of how many processors are available.

Consequently, programs with smaller serial fractions scale more effectively. Reducing the serial portion is essential for achieving high parallel performance, especially when many processors are available.

These are theoretical factors. In real systems, communication overhead, synchronization, memory contention, and load imbalance can also reduce the speedup.

Task 2: Gustafson's Law

6. For s = 0.05, compute the scaled speedup S(p) = s + p(1-s) for the same p values.

Gustafson's law: S(p) = s + p(1-s)

Where:

- S(p) = speedup using p processors
- s = serial fraction of the parallel execution time ➔ s = 0.05
- (1-s) = parallel fraction of the parallel execution time ➔ 1 – s = 0.95
- p = number of processors ➔ p = 1, 2, 4, 8, 16, 64, 1024

Substitute: S(p) = 0.05 + 0.95p

$S(1) = 1$

$S(2) = 1.95$

$S(4) = 0.05 + 4(0.95) = 3.85$

$S(8) = 0.05 + 8(0.95) = 7.65$

$S(16) = 0.05 + 16(0.95) = 15.25$

$S(64) = 0.05 + 64(0.95) = 60.85$

$S(1024) = 0.05 + 1024(0.95) = 972.85$

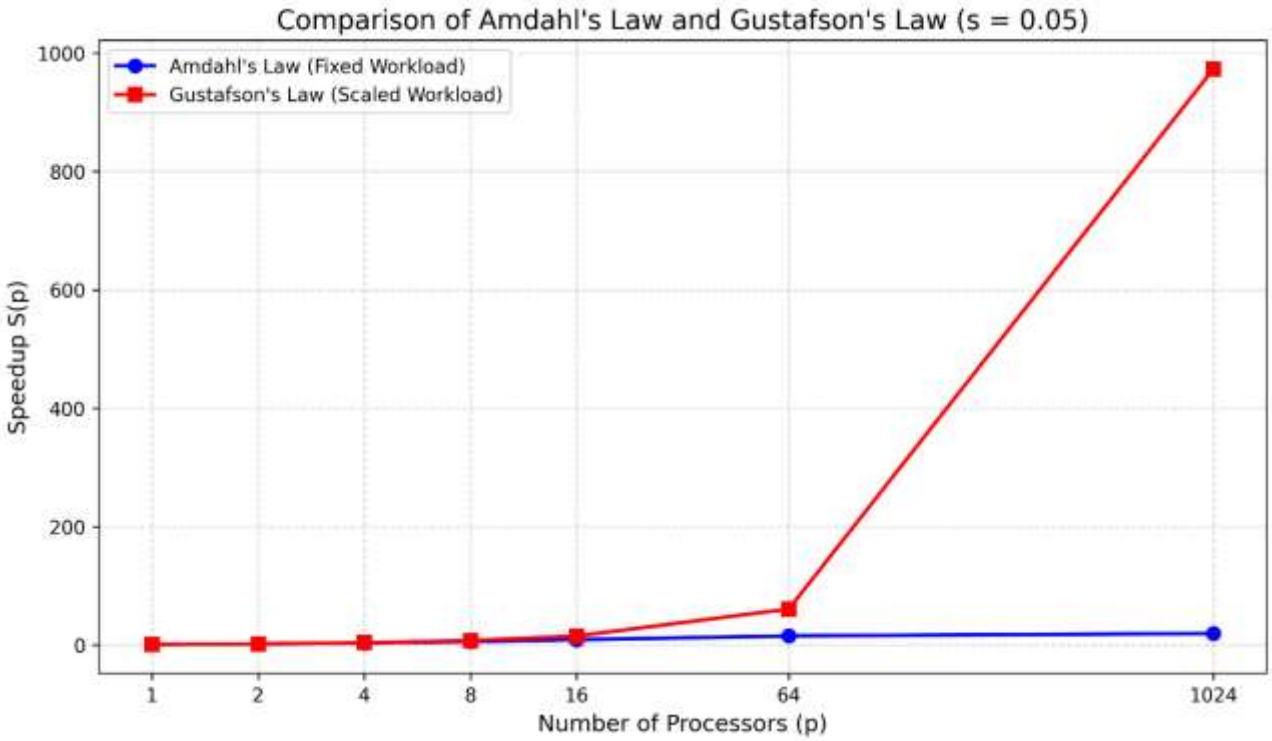
Dalya Mohammad Hassan
220253121

The results show that Gustafson's law shows an approximately linear increase in scaled speedup as more processors are used.

At 1024 processors, the scaled speedup is 972.85, which is approximately 95% of the ideal speedup of 1024.

Gustafson's law does not impose a fixed maximum speedup limit as p increases, provided its constant-fraction scaling assumptions remain valid.

7. Plot Gustafson's speedup alongside Amdahl's speedup on the same graph.



Amdahl's law reaches a maximum theoretical speedup of 20, whereas Gustafson's law shows continued growth as the number of processors increases.

8. Explain in your own words why the two laws give different results.

The main difference between Amdahl's law and Gustafson's law is because of how they treat the workload as the number of processors increases.

- Amdahl's Law: (fixed workload)

  Amdahl's law assumes that the total problem size remains constant when additional processors are added. The parallelizable portion executes faster as the processor count increases, but the serial portion requires the same amount of time.

Consequently, the performance improvement eventually becomes limited by the serial fraction.

For example, when the serial fraction is 5%, the theoretical maximum speedup is s = 1/0.05 = 20.

Even if the number of processors increases indefinitely, the theoretical speedup cannot exceed 20 under this fixed-workload model.

- Gustafson's law: (scaled workload)

  Gustafson's law considers that the problem size grows as more processors become available. The additional processing capacity is used to perform more parallel work in a comparable parallel execution time.

  With a serial fraction of 5% in the parallel execution-time model, the scaled maximum speedup is s = 972.85.

  This shows that increasing the problem size can allow a parallel system to benefit from additional processors rather than being constrained by a fixed amount of parallel work.

In summary, Amdahl's law and Gustafson's law produce different speedup predictions because they assume different workload-scaling conditions. Amdahl's law considers a fixed problem size and shows that the serial fraction limits the maximum possible speedup. Gustafson's law considers a growing problem size and shows that additional processors can be used to solve larger problems efficiently. So, Amdahl's law is more appropriate for evaluating the speedup of a fixed workload, while Gustafson's law is more suitable for evaluating scaled workloads where the amount of parallel computation increases with available processing resources.

9. Give one real-world scenario where each law is more appropriate.

Scenario 1 for Amdahl's Law: Processing a fixed collection of digital documents

Consider a digital library that needs to perform optical character recognition (OCR) on a fixed collection of 100,000 scanned historical documents.

The workload consists of two parts:

Serial processing (5%): Initial configuration, preparation, and other steps that must be completed sequentially.

Parallel processing (95%): OCR operations on independent document pages, which can be distributed across multiple processors.

Because the number of documents remains constant, increasing the number of processors reduces the processing time of the parallel portion without increasing the total amount of work.

Assume that the processing time on one processor is 100 hours. Using Amdahl's law with 16 processors:

$$S(16) = \frac{1}{0.05 + \frac{0.95}{16}} = 9.142857$$

So, the estimated execution time is: $\mathrm{Time} = \frac{100}{9.142857} = 10.9375$ hours

This result assumes ideal parallel distribution without additional overhead.

Amdahl's law is appropriate for this scenario because the document collection remains fixed, and the objective is to minimize the time needed to complete the same workload.

Scenario 2 for Gustafson's Law: Large-scale satellite image processing

Consider a scientific research organization processing satellite images using a parallel computing cluster.

As the organization increases its available computing resources, it also increases the amount of satellite imagery analyzed to enable higher spatial resolution or coverage of larger geographic areas.

For this scenario:

Serial processing (5%): Sequential operations such as initialization and final coordination.

Parallel processing (95%): Independent image-processing tasks distributed across processors.

Assume that the organization increases its processing resources from one processor to 64 processors while scaling the computational workload.

Using Gustafson's Law: $S(64) = 0.05 + 64(0.95) = 60.85$

Under the model assumptions, this means the parallel system performs a scaled workload equivalent to approximately 60.85 times the one-processor work capacity over the corresponding reference execution time.

Gustafson's law is appropriate for this scenario because the objective is not only to complete one fixed dataset faster, but to process more data as additional computational resources are added.

Task 3: Karp–Flatt Metric

You measured the following speedups for a program:

p	T(p) (s)	S(p)
1	120.0	1.00
2	65.0	1.85
4	35.0	3.43
8	20.0	6.00
16	14.0	8.57

Dalya Mohammad Hassan
220253121

10. Compute S(p) from the times and verify the table.

$$S(p) = \frac{T(1)}{T(p)}$$

Where:

- S(p) = speedup using p processors
- T(1) = execution time using one processor ➔ $T(1) = 120$ seconds
- T(p) = execution time using p processors

So $S(p) = \frac{120}{T(p)}$

$$S(1) = \frac{120}{120} = 1$$

$$S(2) = \frac{120}{65} = 1.846154 \approx 1.85$$

$$S(4) = \frac{120}{35} = 3.428571 \approx 3.43$$

$$S(8) = \frac{120}{20} = 6$$

$$S(16) = \frac{120}{14} = 8.571429 \approx 8.57$$

Verify the table: all calculated speedup values agree with the table after rounding to two decimal places.

The observed speedup increases from 1.00 at one processor to approximately 8.57 at 16 processors. However, this is less than the ideal linear speedup of 16, which indicates that the program does not achieve perfect parallel efficiency.

11. Compute the Karp–Flatt metric e for p = 2, 4, 8, 16.

$$e(p) = \frac{\frac{1}{S(p)} - \frac{1}{p}}{1 - \frac{1}{p}}$$

Where:

- e(p) = experimentally determined serial fraction
- S(p) = measured speedup
- p = number of processors

Substitute:

$$e(2) = \frac{\frac{1}{1.846154} - \frac{1}{2}}{1 - \frac{1}{2}} = \frac{0.04166667}{\frac{1}{2}} = 0.0833333 = 8.3333\%$$

$$e(4) = \frac{\frac{1}{3.428571} - \frac{1}{4}}{1 - \frac{1}{4}} = \frac{0.04166667}{\frac{3}{4}} = 0.0555556 = 5.5556\%$$

$$e(8) = \frac{\frac{1}{6} - \frac{1}{8}}{1 - \frac{1}{8}} = \frac{0.04166667}{\frac{7}{8}} = 0.047619 = 4.7619\%$$

$$e(16) = \frac{\frac{1}{8.571429} - \frac{1}{16}}{1 - \frac{1}{16}} = \frac{0.05416667}{\frac{15}{16}} = 0.0577778 = 5.7778\%$$

12. Is the serial fraction constant? What does this tell you?

The values of the serial fraction are not identical when increasing the number of processors p. This means that the serial fraction e is not constant.

The Karp–Flatt metric is not constant across the measured processors number. It decreases from 8.33% for two processors to 4.76% for eight processors. Then, it increases to 5.78% at 16 processors. This behavior means that the program's effective serial fraction varies with processor count. The initial decrease may reflect favorable parallel execution or changes in overhead, while the increase at 16 processors means that there are additional scalability limitations. However, execution times alone are insufficient to identify the exact causes. These results show that a single fixed serial fraction does not describe the observed performance alone.

13. Predict the efficiency at p = 32.

As we are not provided with a measured execution time at 32 processors, we will make an assumption. We can estimate the efficiency using the Karp–Flatt metric and Amdahl's Law $e(p) = \frac{S(p)}{p}$.

Where:

- e(p) = parallel efficiency
- S(p) = Speedup
- p = Number of processors

To predict e(32), we first need to estimate S(32) ➔ Because the e values are not constant, there is no guaranteed prediction at 32 processors.

Dalya Mohammad Hassan
220253121

For a simple prediction, we will assume that the most recently observed effective serial fraction remains constant when the number of processors increases from 16 to 32. So, $e(32) \approx e(16) = 0.0577778$

Now, we will estimate speedup using Amdahl's law at p = 32:

$$S(32) = \frac{1}{0.057777778 + \frac{1-0.057777778}{32}} = \frac{1}{0.057777778 + 0.029444444} = \approx 11.464968$$

Next, we compute the efficiency: $e(p) = \frac{S(p)}{p}$

$$e(32) = \frac{11.464968}{32} \approx 0.358280 = 35.828\%$$

Task 4: Strong vs. Weak Scaling

You are given the following measured times for a program solving a fixed problem (strong scaling) and a problem scaled per processor (weak scaling):

p	Strong T(p) (s)	Weak T(p) (s)
1	100	100
2	55	102
4	30	105
8	18	110
16	12	120

14. Compute the strong-scaling efficiency E(p) = T(1) / (p · T(p)) for each p.

- $E_{strong}(p)$ = strong-scaling efficiency
- T(1) = execution time using one processor for the fixed workload
- T(p) = execution time using p processors for the same workload
- p = number of processors

Substitute:

$$E_{strong}(1) = \frac{100}{1 \times 100} = 1 = 100\%$$

$$E_{strong}(2) = \frac{100}{2 \times 55} = 0.909091 \approx 90.91\%$$

$$E_{strong}(4) = \frac{100}{4 \times 30} = 0.833333 \approx 83.33\%$$

$$E_{strong}(8) = \frac{100}{8 \times 18} = 0.694444 \approx 69.44\%$$

$$E_{strong}(16) = \frac{100}{16 \times 12} = 0.520833 \approx 52.08\%$$

The results show that strong-scaling efficiency decreases as the number of processors increases.

Dalya Mohammad Hassan
220253121

At 2 processors, the program achieves approximately 90.91% efficiency. At 4 processors, efficiency decreases to 83.33%. However, at 8 processors, it falls to 69.44%, and at 16 processors, it decreases further to 52.08%.

Although execution time improves from 100 seconds to 12 seconds, the program does not achieve ideal linear speedup. This means that efficiency decreases as more processors are added to execute the same fixed workload.

15. Compute the weak-scaling efficiency E(p) = T(1) / T(p) for each p.
    - $E_{weak}(p)$ = strong-scaling efficiency
    - T(1) = execution time using one processor
    - T(p) = execution time using p processors with a scaled problem size
    - p = number of processors

Substitute:

$$E_{weak}(1) = \frac{100}{100} = 1 = 100\%$$

$$E_{weak}(2) = \frac{100}{102} = 0.980392 \approx 98.04\%$$

$$E_{weak}(4) = \frac{100}{105} = 0.952381 \approx 95.24\%$$

$$E_{weak}(8) = \frac{100}{110} = 0.909091 \approx 90.91\%$$

$$E_{weak}(16) = \frac{100}{120} = 0.833333 \approx 83.33\%$$

The results show that weak-scaling efficiency decreases gradually as the number of processors increases.

At 2 processors, the program has approximately 98.04% efficiency, which means that it can process a doubled workload with a small increase in execution time.

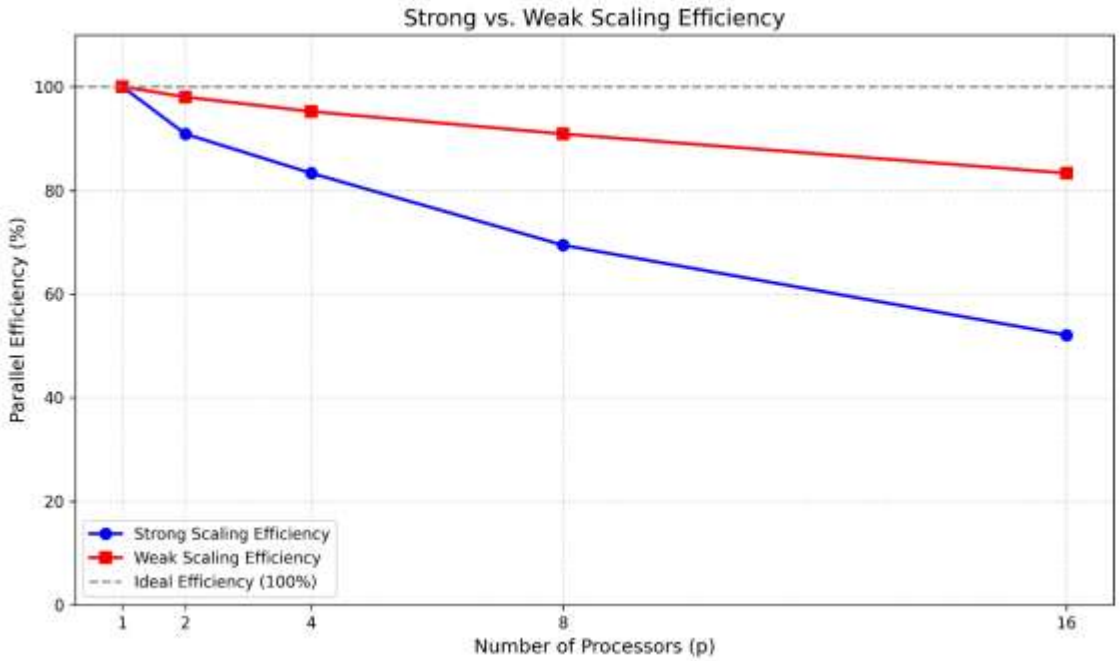
At 16 processors, the workload is assumed to have increased proportionally by a factor of 16, while execution time increases from 100 to 120 seconds. The efficiency is 83.33%.

This indicates that the system has relatively good performance when the problem size increases with the processor count.

16. Plot both efficiencies on the same graph.

```
TASK 4: STRONG VS. WEAK SCALING
-------------------------------------------------------
Processors |   Strong (%) |   Weak (%)
-------------------------------------------------------
         1 |     100.0000 |   100.0000
         2 |      90.9091 |    98.0392
         4 |      83.3333 |    95.2381
         8 |      69.4444 |    90.9091
        16 |      52.0833 |    83.3333
```

Dalya Mohammad Hassan
220253121



The comparison graph shows that both strong-scaling and weak-scaling efficiencies decrease as the number of processors increases. However, the strong-scaling efficiency decreases more rapidly. The graph also shows that weak-scaling efficiency remains higher than strong-scaling efficiency when the number of processors is greater than one.

At 4 processors, strong-scaling efficiency is approximately 83.33%, compared with 95.24% for weak scaling.

At 16 processors, strong-scaling efficiency decreases to 52.08%, while weak-scaling efficiency remains at 83.33%.

These observations show that the program loses more efficiency when distributing a fixed workload across additional processors than when increasing the workload proportionally with the number of processors.

17. Which scaling type is this program better at? Justify your answer.

First, let's compare the performance:

Strong scaling keeps the problem size fixed while increasing the number of processors. For this program:

- Execution time decreases from 100 seconds to 12 seconds.
- The measured speedup at 16 processors is 8.33.
- Efficiency decreases from 100% to 52.08%.

Weak scaling increases the problem size proportionally with the number of processors. For this program:

- The workload increases as processors are added.
- Execution time increases from 100 seconds to 120 seconds.
- Efficiency decreases from 100% to 83.33%.

Next, let's compare efficiency with 16 processors:

$E_{\text{strong}}(16) = 0.520833 \approx 52.08\%$

$E_{weak}(16) = 0.833333 \approx 83.33\%$

The efficiency difference is: $\Delta E = 83.3333\% - 52.0833\% = 31.25$ percentage points

Weak-scaling efficiency is 31.25 percentage points higher than strong-scaling efficiency at 16 processors.

The program shows that weak-scaling performance is better than strong-scaling performance. At 16 processors, weak-scaling efficiency remains at approximately 83.33%, whereas strong-scaling efficiency decreases to 52.08%. This difference indicates that the program is more effective at processing larger workloads as additional processors are added than at accelerating a fixed workload. The strong-scaling results show diminishing efficiency as processor count increases, while weak scaling keeps execution time relatively stable despite the growing workload. So, weak scaling is the more effective scaling approach for this program over the measured processor range.

18. What could limit the strong scaling?

Strong scaling may be limited by many factors, including serial execution, communication overhead, synchronization costs, load imbalance, and contention for shared hardware resources. According to Amdahl's Law, the serial portion of a fixed workload establishes a theoretical limit on speedup. Additionally, as more processors are introduced, communication and synchronization overhead may become significant relative to the useful computational work assigned to each processor. Uneven workload distribution and memory-resource contention can also reduce processor utilization. These factors provide possible explanations for the observed decrease in strong-scaling efficiency from 100% at one processor to 52.08% at 16 processors. However, additional profiling measurements would be required to identify which specific performance bottleneck is responsible.

Task 5: Scaling Analysis and Discussion

You are provided a strong-scaling dataset with 5 repeated measurements per processor count (see data.csv provided in class). Complete the following:

You can use this dataset

p	t_1	t_2	t_3	t_4	t_5
1	240.0	238.5	241.2	239.8	240.5
2	122.5	121.0	123.2	121.8	122.0
4	63.0	62.2	63.8	62.5	63.1
8	33.5	32.8	34.0	33.1	33.6
16	18.9	18.4	19.2	18.6	18.8
32	12.5	12.1	12.9	12.3	12.6
64	9.8	9.4	10.1	9.6	9.9
128	8.7	8.3	9.0	8.5	8.8

19. Compute the mean and standard deviation of time at each processor count.

The mean execution time is the average of the 5 repeated measurements. $\overline{T}(p) = \frac{1}{n}\sum_{i=1}^{n} t_i\,(p)$

Where:

- $\overline{T}(p)$ = mean execution time for p processors
- $t_i(p)$ = Execution time of measurement i
- $n = 5$ = number of measurements

Standard deviation:

$SD(p) = \sqrt{\frac{\sum_{i=1}^{n}\left(t_i(p)-\overline{T}(p)\right)^2}{n-1}}$ ➔ n = 5

$SD(p) = \sqrt{\frac{\sum_{i=1}^{5}\left(t_i(p)-\overline{T}(p)\right)^2}{4}}$

Substitute:

$\overline{T}(1) = \frac{240 + 238.5 + 241.2 + 239.8 + 240.5}{5} = \frac{1200}{5}$ ==> $\overline{T}(1) = 240$ s

$SD(1) = \sqrt{\frac{0^2 + (-1.5)^2 + 1.2^2 + (-0.2)^2 + 0.5^2}{4}} = \sqrt{\frac{3.98}{4}}$ ==> $SD(1) = 0.9975$ s

$\overline{T}(2) = \frac{122.5 + 121 + 123.2 + 121.8 + 122}{5} = \frac{610.5}{5}$ ==> $\overline{T}(2) = 122.1$ s

Dalya Mohammad Hassan
220253121

$$SD(2) = \sqrt{\frac{0.4^2 + (-1.1)^2 + 1.1^2 + (-0.3)^2 + (-0.1)^2}{4}} = \sqrt{\frac{2.68}{4}} \quad ==> \quad SD(2) = 0.8185 \text{ s}$$

$$\overline{T}(4) = \frac{63 + 62.2 + 63.8 + 62.5 + 63.1}{5} = \frac{314.6}{5} \quad ==> \quad \overline{T}(4) = 62.92 \text{ s}$$

$$SD(4) = \sqrt{\frac{0.08^2 + (-0.72)^2 + 0.88^2 + (-0.42)^2 + 0.18^2}{4}} = \sqrt{\frac{1.508}{4}} \quad ==> \quad SD(4) = 0.614 \text{ s}$$

$$\overline{T}(8) = \frac{33.5 + 32.8 + 34 + 33.1 + 33.6}{5} = \frac{167}{5} \quad ==> \quad \overline{T}(8) = 33.4 \text{ s}$$

$$SD(8) = \sqrt{\frac{0.1^2 + (-0.6)^2 + 0.6^2 + (-0.3)^2 + 0.2^2}{4}} = \sqrt{\frac{0.86}{4}} \quad ==> \quad SD(8) = 0.4637 \text{ s}$$

$$\overline{T}(16) = \frac{18.9 + 18.4 + 19.2 + 18.6 + 18.8}{5} = \frac{93.9}{5} \quad ==> \quad \overline{T}(16) = 18.78 \text{ s}$$

$$SD(16) = \sqrt{\frac{0.12^2 + (-0.38)^2 + 0.42^2 + (-0.18)^2 + 0.02^2}{4}} = \sqrt{\frac{0.368}{4}} \quad ==> \quad SD(16) = 0.3033 \text{ s}$$

$$\overline{T}(32) = \frac{12.5 + 12.1 + 12.9 + 12.3 + 12.6}{5} = \frac{62.4}{5} \quad ==> \quad \overline{T}(32) = 12.48 \text{ s}$$

$$SD(32) = \sqrt{\frac{0.02^2 + (-0.38)^2 + 0.42^2 + (-0.18)^2 + 0.12^2}{4}} = \sqrt{\frac{0.368}{4}} \quad ==> \quad SD(32) = 0.3033 \mp \text{ s}$$

$$\overline{T}(64) = \frac{9.8 + 9.4 + 10.1 + 9.6 + 9.9}{5} = \frac{48.8}{5} \quad ==> \quad \overline{T}(64) = 9.76 \text{ s}$$

$$SD(64) = \sqrt{\frac{0.04^2 + (-0.36)^2 + 0.34^2 + (-0.16)^2 + 0.14^2}{4}} = \sqrt{\frac{0.292}{4}} \quad ==> \quad SD(64) = 0.2702 \text{ s}$$

$$\overline{T}(128) = \frac{8.7 + 8.3 + 9 + 8.5 + 8.8}{5} = \frac{43.3}{5} \quad ==> \quad \overline{T}(128) = 8.66 \text{ s}$$

$$SD(128) = \sqrt{\frac{0.04^2 + (-0.36)^2 + 0.34^2 + (-0.16)^2 + 0.14^2}{4}} = \sqrt{\frac{0.292}{4}} \quad ==> \quad SD(128) = 0.2702 \text{ s}$$

Dalya Mohammad Hassan
220253121

The mean execution time decreases as the number of processors increases. However, the absolute decrease in execution time becomes smaller at higher processor counts.

The sample standard deviations are relatively small compared with the corresponding means, which means that the five repeated measurements at each processor count exhibit limited variability.

20. Compute speedup and efficiency using the mean time.

Speedup:

$$S(p) = \frac{\overline{T}(1)}{\overline{T}(p)}$$

And $\overline{T}(1) = 240.00$ s ➔ $S(p) = \frac{240}{\overline{T}(p)}$

Efficiency:

$$E(p) = \frac{S(p)}{p} = \frac{\overline{T}(1)}{p\overline{T}(p)}$$

Substitute:

$$S(1) = \frac{240}{240} = 1$$

$$E(1) = \frac{1}{1} = 1 = 100\%$$

$$S(2) = \frac{240}{122.1} = 1.9656$$

$$E(2) = \frac{1.9656}{2} = 0.9828 \approx 98.28\%$$

$$S(4) = \frac{240}{62.92} = 3.8144$$

$$E(4) = \frac{3.8144}{4} = 0.9536 \approx 95.36\%$$

$$S(8) = \frac{240}{33.4} = 7.1856$$

Dalya Mohammad Hassan
220253121

$$E(8) = \frac{7.1856}{8} = 0.8982 \approx 89.82\%$$

$$S(16) = \frac{240}{18.78} = 12.7796$$

$$E(16) = \frac{12.7796}{16} = 0.7987 \approx 79.87\%$$

$$S(32) = \frac{240}{12.48} = 19.2308$$

$$E(32) = \frac{19.2308}{32} = 0.6010 \approx 60.10\%$$

$$S(64) = \frac{240}{9.76} = 24.5902$$

$$E(64) = \frac{24.5902}{64} = 0.3842 \approx 38.42\%$$

$$S(128) = \frac{240}{8.66} = 27.7136$$

$$E(128) = \frac{27.7136}{128} = 0.2165 \approx 21.65\%$$

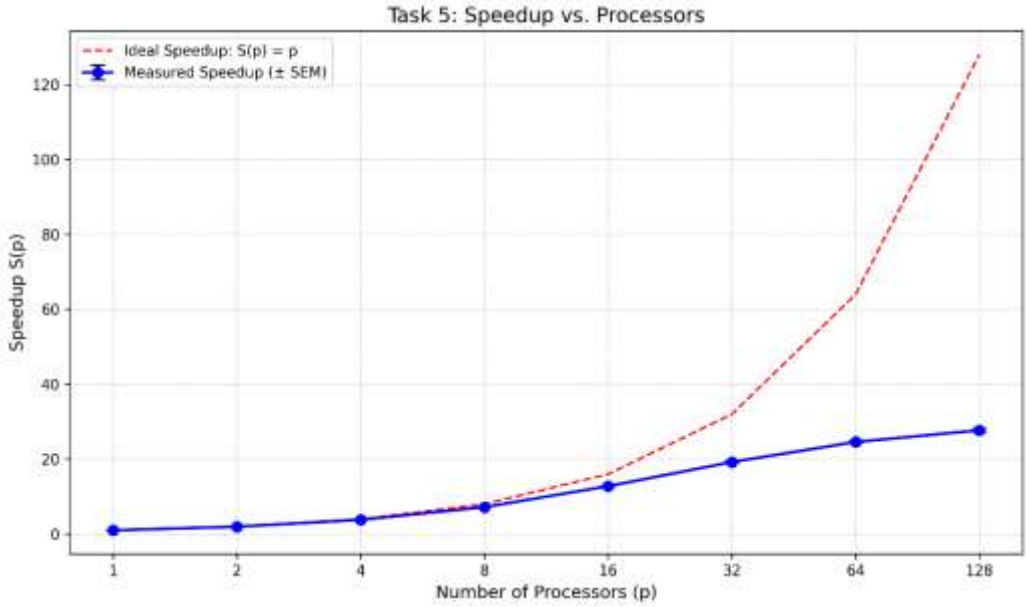
The speedup increases as more processors are used, reaching 27.71 with 128 processors. However, efficiency decreases, especially after 16 processors.

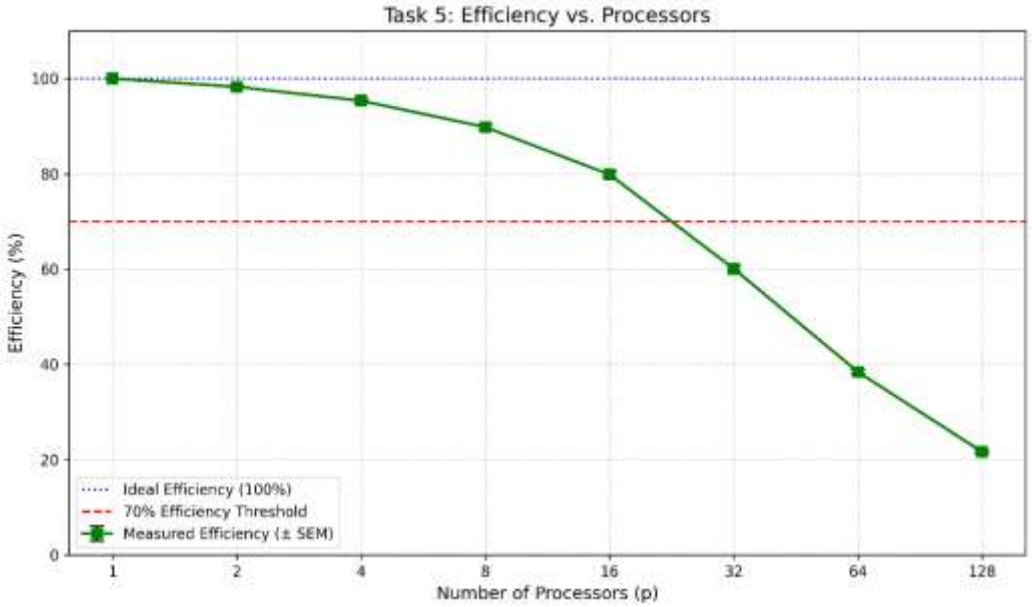
This supports the idea that faster execution does not necessarily mean more efficient utilization of processors.

At 128 processors, the program is approximately 27.71 times faster than the single-processor execution, but its efficiency is only 21.65%. This indicates substantial diminishing returns from additional processors.

Dalya Mohammad Hassan
220253121

21. Plot speedup vs. p and efficiency vs. p with error bars.





The speedup and efficiency plots show that the application initially exhibits strong parallel scaling, with efficiency above 90% up to 4 processors. As the number of processors increases, the speedup continues to improve, but the improvement becomes increasingly sublinear. Efficiency falls sharply after 16 processors, reaching 21.65% at 128 processors. The propagated error bars indicate that the

observed scaling deterioration is much larger than the estimated uncertainty related to the repeated measurements.

22. Identify the processor count at which efficiency drops below 70%.

$$E(p) = \frac{\overline{T}(1)}{p\overline{T}(p)}$$

We need to find the first measured processor count that satisfies $E(p) < 0.70$
Let's check efficiency at 16 processors: $E(16) = \frac{240}{16 \times 18.78} = \frac{240}{300.48} = 0.7987 = 79.87\%$
As $79.87\% > 70\%$ ➔ This means efficiency is still above the threshold.

Let's check efficiency at 32 processors: $E(32) = \frac{240}{32 \times 12.48} = \frac{240}{399.36} = 0.601 = 60.1\%$
As $60.10\% < 70\%$ ➔ This means efficiency is below the threshold.

So, the first measured processor count at which parallel efficiency falls below 70% is 32 processors, where efficiency is 60.10%. This indicates a significant reduction in processor utilization under the strong-scaling efficiency metric. Based on the measured values, 16 processors are the largest tested configuration that maintains efficiency above 70%.

This finding is supported by code in the provided Python file.

23. List at least three bottlenecks that could cause this drop.

The decrease in strong-scaling efficiency may be limited due to many bottlenecks. First, serial execution limits the maximum achievable speedup according to Amdahl's Law. Second, communication and synchronization overhead may become increasingly significant as the workload is distributed across more processors. Third, load imbalance can cause some processors to remain idle while others complete their assigned tasks. Additionally, memory bandwidth contention and cache-coherence overhead can reduce performance in shared-memory architectures. These factors may explain the drop in efficiency from 79.87% at 16 processors to 60.10% at 32 processors and 21.65% at 128 processors. Additional experiments would be needed to determine which factors are responsible.

24. Suggest two improvements to increase scalability.

Improvement 1: Reduce communication and synchronization overhead.
Parallel programs should minimize unnecessary data exchanges and synchronization operations by:

- Combining multiple small messages into larger communication operations.
- Avoiding unnecessary global synchronization barriers.
- Reducing contention for shared variables.
- Overlapping communication with useful computation where possible.
- Using thread-local or process-local data to reduce coordination.

This can reduce communication and synchronization overhead and allow processors to spend more time performing useful computations instead of waiting for data.
This can improve strong-scaling efficiency, especially when the workload assigned to each processor becomes smaller.

Improvement 2: Improve load balancing and data locality

Work should be distributed so that processors receive approximately equal amounts of computational work, while data placement minimizes unnecessary memory access by:

- Using dynamic or guided scheduling for workloads with uneven task execution times.
- Partitioning work according to computational cost rather than only task count.
- Keeping frequently accessed data close to the processors that use it.
- Applying cache-friendly data layouts and blocking techniques.
- Using NUMA-aware memory placement and processor affinity where appropriate.

Implementing better load balancing reduces idle time, while improved data locality reduces memory-access overhead.

These changes can improve processor utilization and increase scalability.

Dalya Mohammad Hassan
220253121